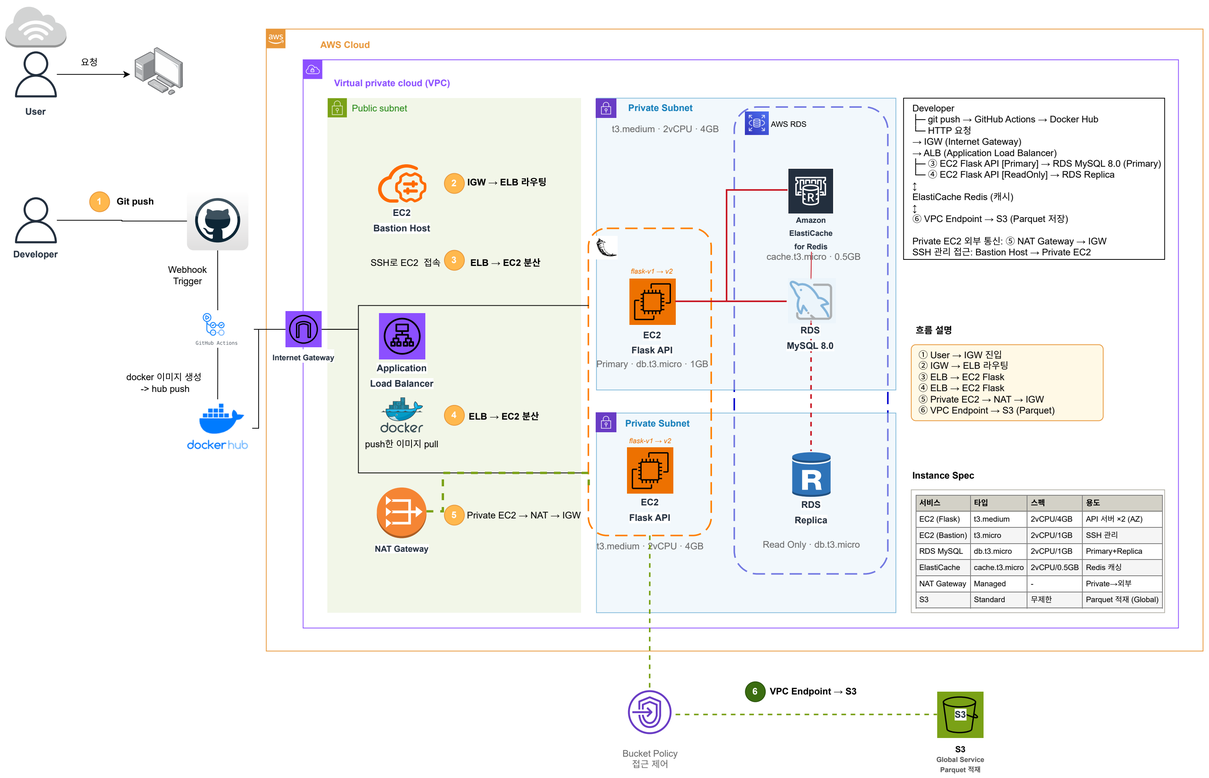
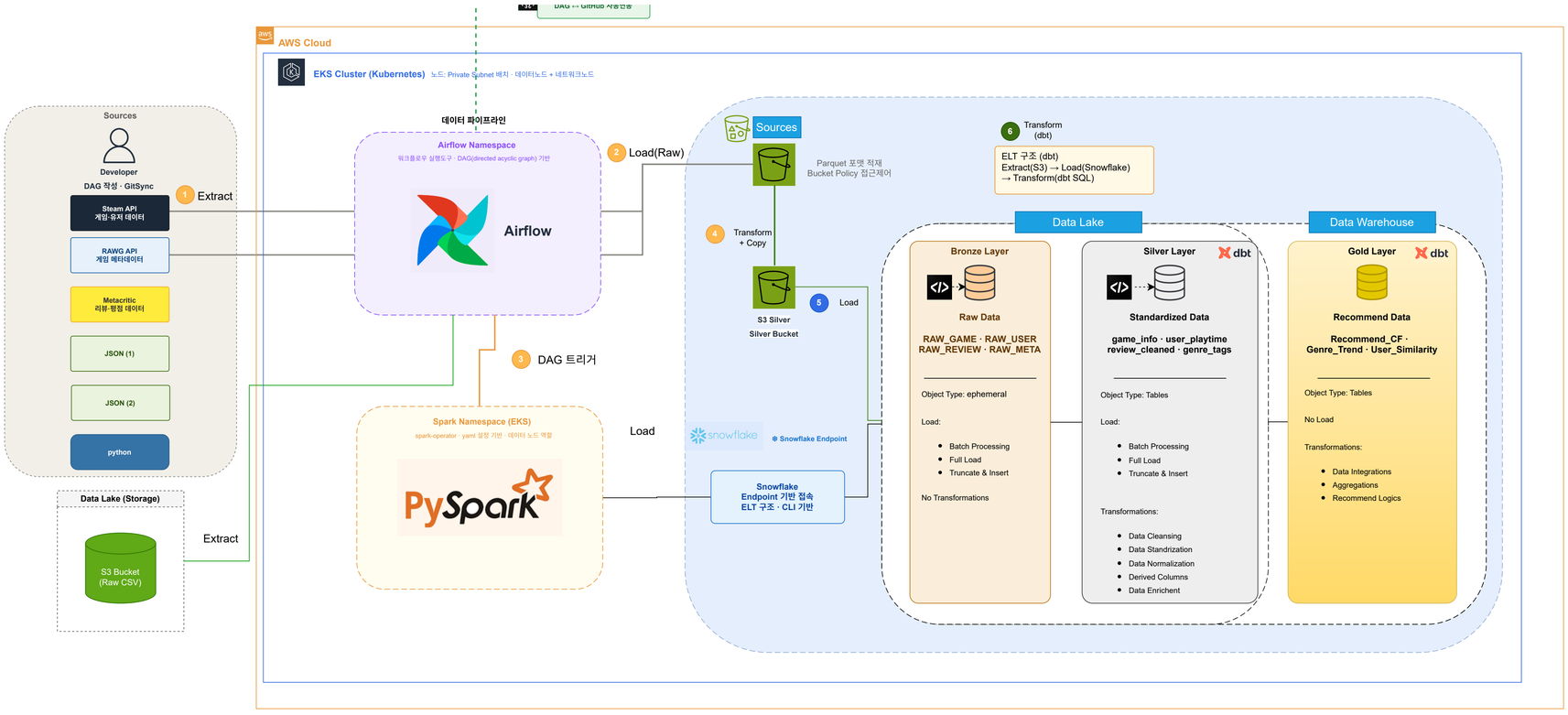


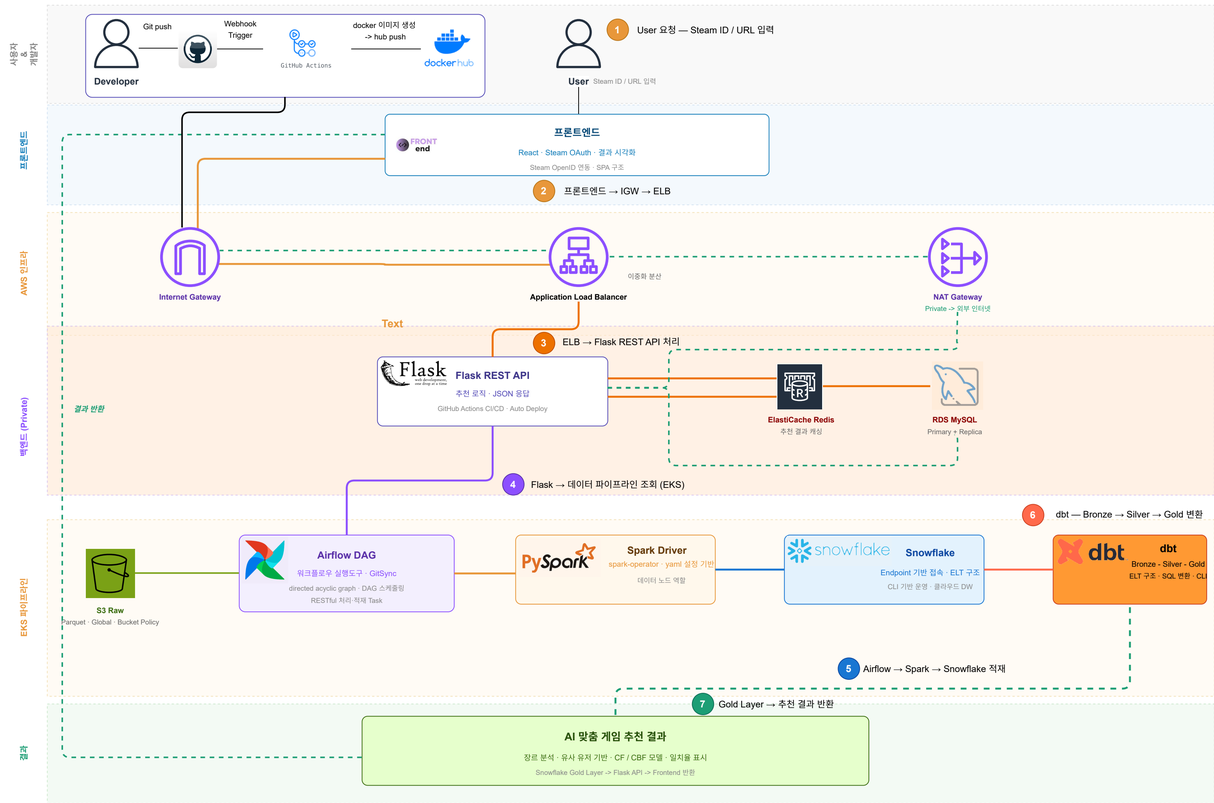
서비스	타입	스펙	용도
EC2 (Flask)	t3.medium	2vCPU/4GB	API 서버 ×2 (AZ)
EC2 (Bastion)	t3.micro	2vCPU/1GB	SSH 관리
RDS MySQL	db.t3.micro	2vCPU/1GB	Primary+Replica
ElastiCache	cache.t3.micro	2vCPU/0.5GB	Redis 캐싱
NAT Gateway	Managed	-	Private→외부
S3	Standard	무제한	Parquet 적재 (Global)

번호	라벨	설명
1	개발자 요청	• `git push` → GitHub → **GitHub Actions** 트리거 • `docker 이미지 생성 → hub push` → **Docker Hub** 업로드
2	IGW → ELB 라우팅	Internet Gateway가 외부 트래픽을 받아 Public Subnet 내 **Application Load Balancer**로 전달 • EC2 Bastion Host가 동일 Public Subnet에 위치 (SSH 관리용)
3	ELB → EC2 분산	ALB가 Private Subnet 상단의 **EC2 Flask API (Primary)** 로 트래픽 분산 • t3.medium · 2vCPU · 4GB • Primary RDS MySQL 8.0과 연결
4	ELB → EC2 분산	Docker Hub에서 이미지를 pull 후 Private Subnet 하단의 **EC2 Flask API (Read Only)** 로 분산 • t3.medium · 2vCPU · 4GB • RDS Replica와 연결
5	Private EC2 → NAT → IGW (외부 출구)	Private EC2가 외부 통신 필요 시 **NAT Gateway**를 경유해 IGW로 나감 • 인바운드 차단, 아웃바운드만 허용
6	VPC Endpoint → S3	Private Subnet → 인터넷 우회 없이 **VPC Endpoint**를 통해 S3 직접 연결 • Bucket Policy로 접근 제어 • S3 Global Service, Parquet 형식 저장

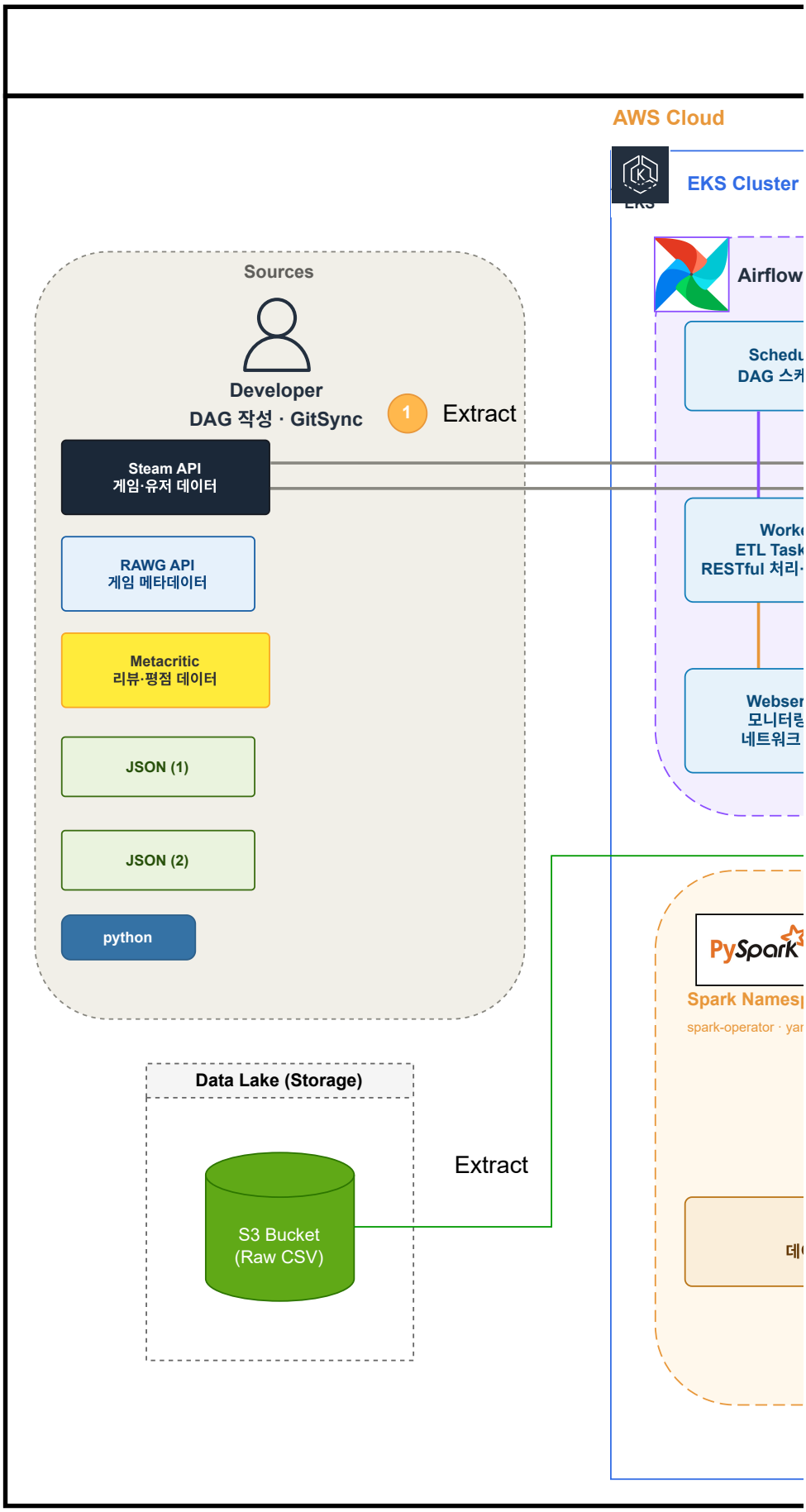
Webserver
모니터링 UI
네트워크 노드
Scheduler
DAG 스케줄링
Worker
ETL Task 실행
RESTful 처리·적재 Task
GitSync
DAG ← GitHub 자동연동

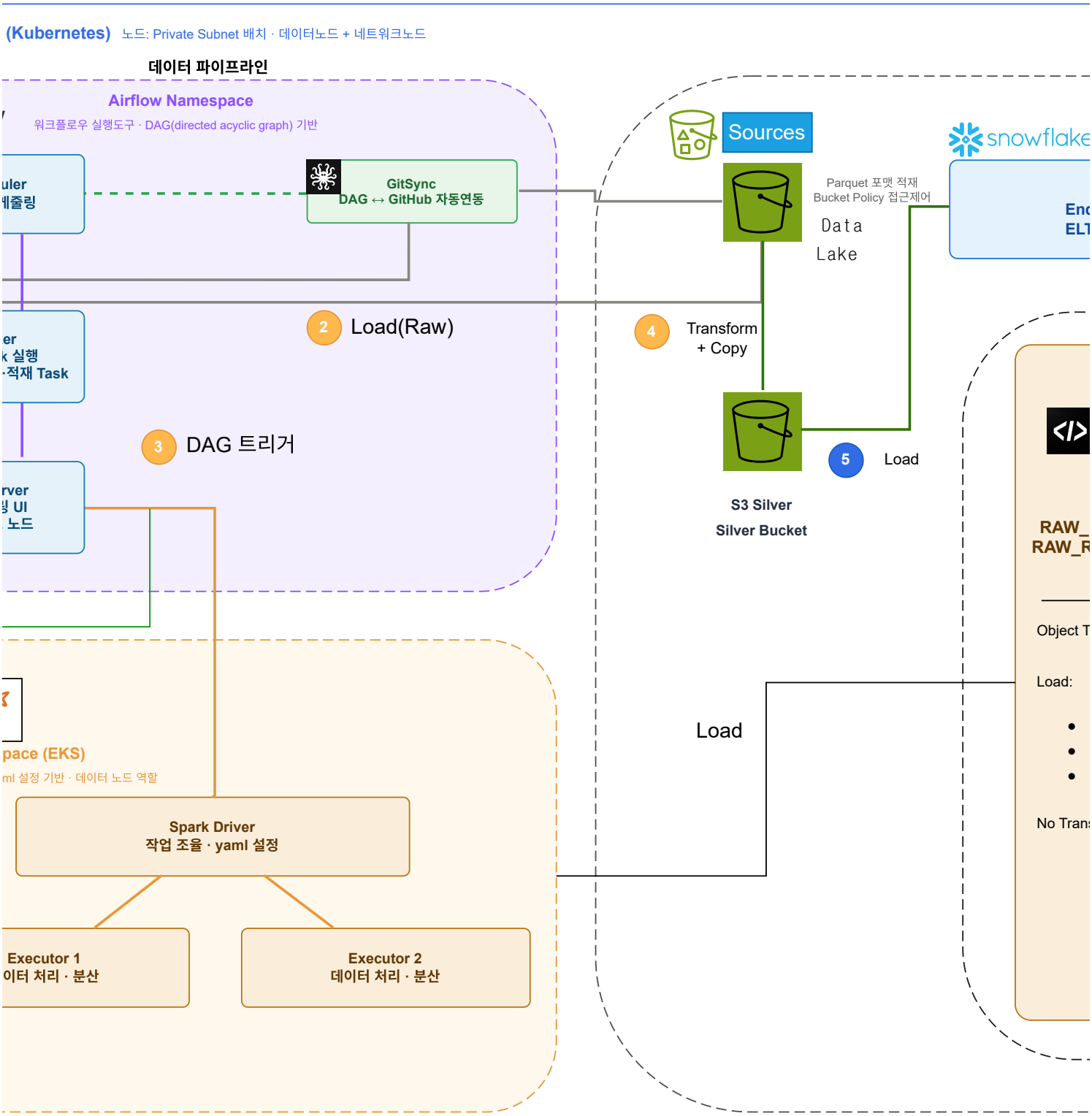


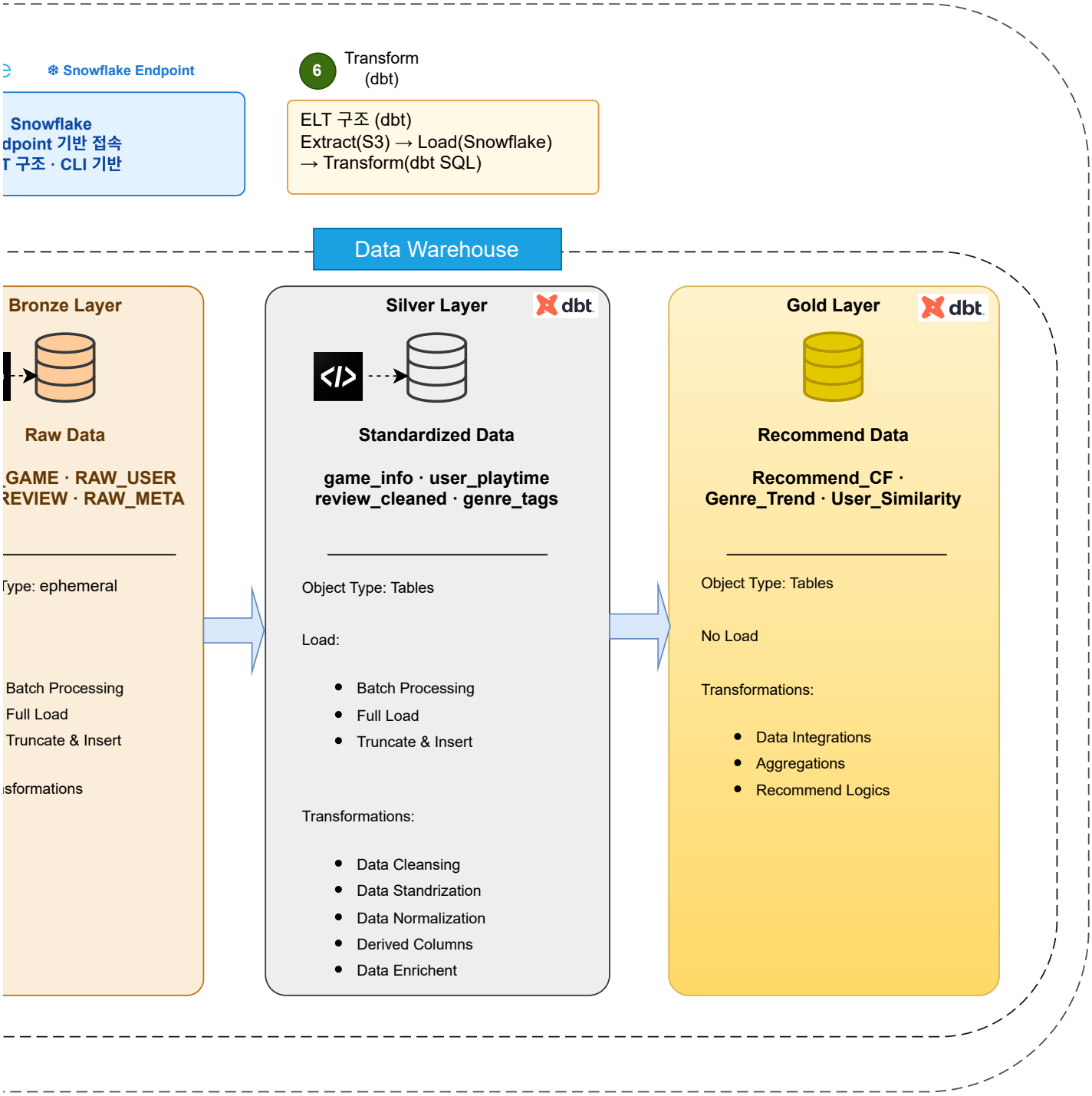
번호	라벨	설명
1	Extract	**데이터 수집** Steam API, RAWG API, Metacritic, JSON 파일에서 Python 스크립트로 원천 데이터를 가져옴
2	**Load (Raw)**	**S3 Bronze Bucket 적재** 수집된 원시 데이터를 **Parquet 포맷**으로 S3 Bronze Bucket에 적재. Bucket Policy로 접근 제어
3	**DAG 트리거**	**워크플로우 스케줄링** Airflow Scheduler가 GitSync로 GitHub에서 DAG를 자동 동기화하고, Worker가 ETL Task를 실행, DAG = directed acyclic graph 기반의 실행 순서 정의 **Spark Driver 작업 조율 · yaml 설정** **→ Executor 1 데이터 처리 · 분산** **→ Executor 2 데이터 처리 · 분산**
4	**Transform + Copy**	**Spark 분산 처리** Airflow Worker가 Spark Driver를 트리거하고, Executor 1·2가 병렬로 데이터를 처리. S3 Bronze → S3 Silver로 Copy & 정제
5	Load	**Snowflake 적재** 정제된 데이터를 Snowflake Endpoint로 Load, ELT 구조이므로 Load 먼저, Transform은 Snowflake 안에서 수행
6	**Transform (dbt)**	**dbt SQL 변환 (Bronze → Silver → Gold)** dbt가 Snowflake 안에서 SQL로 데이터를 변환 • **Bronze** → RAW_GAME, RAW_USER, RAW_REVIEW, RAW_META (ephemeral) • **Silver** → game_info, user_playtime, review_cleaned, genre_tags (table) • **Gold** → Recommend_CF, Genre_Trend, User_Similarity 등 추천 피처 (table)



번호	라벨	설명
1	User 요청 — Steam ID / URL 입력	사용자가 브라우저에서 **Game Finder 웹사이트**에 접속해 Steam ID 또는 프로필 URL을 입력 프론트엔드: React 사용자가 입력하면 Steam API를 통해 해당 유저의 플레이 기록, 보유 게임 목록, 플레이타임 데이터를 가져와서 화면에 장르 분포 차트와 많이 플레이한 게임 목록 view 예시("맞춤 추천작 보기") 버튼을 누르면 백엔드에 추천 요청
2	프론트엔드 → IGW → ELB	사용자의 추천 요청이 HTTP/HTTPS로 발생하면 AWS **Internet Gateway(IGW)** 를 통해 VPC 안으로 진입하고, **Application Load Balancer(ELB)** 가 수신 상태가 좋은 곳으로 라우팅
3	ELB → Flask REST API 처리	ELB가 트래픽을 **Private Subnet의 EC2** 로 전달하면, Docker 컨테이너 안에서 실행 중인 **Flask REST API** 서버가 요청 받음 Flask는 요청을 파싱해서 Steam ID를 추출하고, 필요한 데이터를 조회 이때 **ElastiCache(Redis)** 를 먼저 확인 같은 유저가 이미 추천을 받은 적 있으면 Redis에 캐싱된 결과를 바로 반환해서 응답 속도를 크게 줄임 캐시에 없으면 **RDS MySQL** 에서 유저 정보와 게임 데이터를 조회하고 데이터 파이프라인에 추천 결과를 요청
4	Flask → 데이터 파이프라인 조회 (EKS)	Flask가 **Airflow DAG**에 파이프라인 실행을 요청 Airflow Scheduler는 GitSync로 GitHub에서 DAG 파일을 자동 동기화 Flask의 요청을 받으면 해당 유저의 Steam 데이터 수집부터 추천 결과 생성까지의 전체 파이프라인을 DAG 순서에 따라 실행 각 Task 사이의 의존 관계가 Directed Acyclic Graph로 정의되어 있어서 이전 Task가 성공해야 다음 Task가 실행 Private EC2가 외부 API를 호출해야 할 때는 직접 인터넷으로 나가지 못하고 **NAT Gateway → IGW** 경로로 우회
5	Airflow → Spark → Snowflake 적재	Airflow Worker가 **Spark Driver**를 트리거 -> EKS의 spark-operator가 Executor 1·2를 띄워서 병렬로 데이터를 처리 S3 Raw 버킷에서 해당 유저 관련 Parquet 데이터를 읽어서 Spark가 정제·변환한 뒤 **Snowflake Endpoint**로 Load 인터넷을 거치지 않고 **VPC Endpoint**를 통해 S3에 직접 접근하기 때문에 속도가 빠르고 비용도 절감
6	dbt — Bronze → Silver → Gold 변환	Snowflake 안에 쌓인 데이터를 **dbt**가 SQL로 단계적으로 변환 Bronze(RAW_GAME, RAW_USER 등 원본) → Silver(game_info, user_playtime, genre_tags 등 정제) → Gold(Recommend_CF, Recommend_CBF, Genre_Trend 등 추천 피처)로 올라가면서 데이터가 점점 분석에 최적화 dbt는 CLI 기반으로 동작하고 각 모델을 SQL 파일로 관리해서 버전 관리와 테스트가 가능
7	Gold Layer → 추천 결과 반환	Gold Layer의 최종 피처 테이블을 기반으로 **AI 맞춤 추천 결과**를 생성 **Recommend_CF** (협업 필터링) — 나와 플레이 패턴이 비슷한 유저들이 즐긴 게임을 추천 **Recommend_CBF** (콘텐츠 기반 필터링) — 내가 많이 플레이한 게임의 장르, 태그, 메타데이터가 비슷한 게임을 추천 **Genre_Trend** — 내가 선호하는 장르 기반 트렌드 게임을 추천 이렇게 생성된 결과가 Flask → ELB → 프론트엔드로 반환되고, 사용자는 게임 커버 이미지, 일치율(%), Steam 스토어 바로가기 링크와 함께 맞춤 추천 게임 목록을 확인


AWS Cloud
EKS Cluster
Sources
Developer
DAG 작성 · GitSync
1
Extract
Steam API
게임·유저 데이터
RAWG API
게임 메타데이터
Metacritic
리뷰·평점 데이터
JSON (1)
JSON (2)
python
Airflow
Schedu
DAG 스케
Work
ETL Task
RESTful 처리·
Webser
모니터링
네트워크
PySpark
Spark Namesp
spark-operator · yar
데
Data Lake (Storage)
S3 Bucket
(Raw CSV)
Extract


(Kubernetes) 노드: Private Subnet 배치 · 데이터노드 + 네트워크노드
데이터 파이프라인
Airflow Namespace
워크플로우 실행도구 · DAG(directed acyclic graph) 기반
GitSync
DAG ↔ GitHub 자동연동
2 Load(Raw)
3 DAG 트리거
Sources
snowflake
Parquet 포맷 적재
Bucket Policy 접근제어
Data
Lake
4 Transform
+ Copy
5 Load
S3 Silver
Silver Bucket
Load
Spark Driver
작업 조율 · yaml 설정
Executor 1
Executor 2
데이터 처리 · 분산


Snowflake Endpoint
Snowflake
dpoint 기반 접속
T 구조 · CLI 기반
6
Transform
(dbt)
ELT 구조 (dbt)
Extract(S3) → Load(Snowflake)
→ Transform(dbt SQL)
Data Warehouse
Bronze Layer
Raw Data
GAME · RAW_USER
REVIEW · RAW_META
Type: ephemeral
Batch Processing
Full Load
Truncate & Insert
sformations
Silver Layer
dbt
Standardized Data
game_info · user_playtime
review_cleaned · genre_tags
Object Type: Tables
Load:
Batch Processing
Full Load
Truncate & Insert
Transformations:
Data Cleansing
Data Standrization
Data Normalization
Derived Columns
Data Enrichent
Gold Layer
dbt
Recommend Data
Recommend_CF ·
Genre_Trend · User_Similarity
Object Type: Tables
No Load
Transformations:
Data Integrations
Aggregations
Recommend Logics